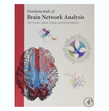
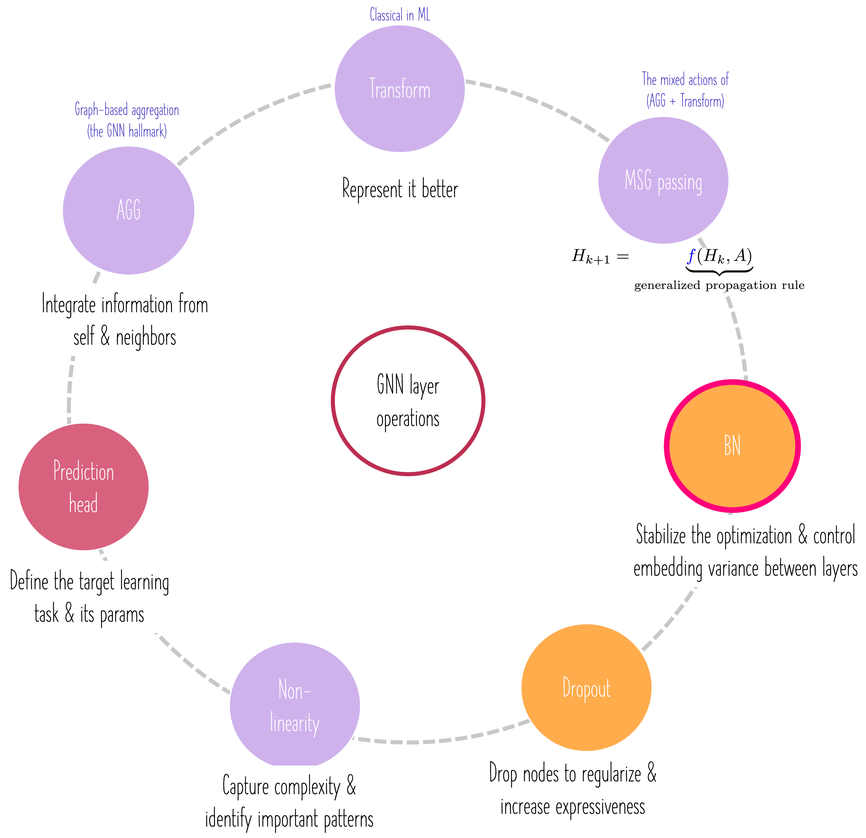
DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks

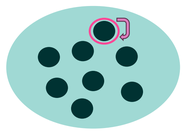


Reminder

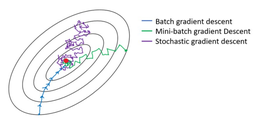
To train a model by optimizing the loss objective, one can adopt one of the following 3 training strategies:

1) Stochastic gradient descent

For each data point:
- Calculate the gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{point}}$

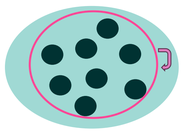


Individual Data Points: In single-point training (or stochastic training), each update is based on the gradient calculated from a single randomly chosen data point.
Frequent Updates: Parameters $\boldsymbol{w}$ are updated frequently after every single point ⟶ $\boldsymbol{w}$ are less stable and can exhibit more variability.
Noisy Updates: The updates can be noisy, leading to more erratic convergence behavior.
Computational Efficiency: It is computationally inefficient especially when dealing with large datasets.



2) Batch gradient descent

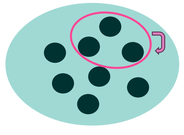
For the whole dataset:
- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{dataset}}$



Full Dataset Usage: In batch training, the entire dataset is used for each training iteration ⟶ the model parameters $\boldsymbol{w}$ are updated only once per epoch (an epoch is a complete pass through the entire training dataset).
Memory Requirements: The entire dataset is loaded into memory ⟶ very memory-intensive, and impractical for large datasets.
Stability: It provides stable error gradients and convergence, as the gradient calculation is based on the entire dataset, leading to a consistent update direction.
Computational Efficiency: Computationally inefficient due to the need to process the entire dataset before making a single update.
Risk of Overfitting: since the model is repeatedly exposed to the entire dataset in the same order (no randomness for regularization).

3) Mini-batch gradient descent

For every batch:
- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{batch}}$



Subset of Dataset: Mini-batch training involves dividing the dataset into smaller subsets or "mini-batches." ⟶ The parameters $\boldsymbol{w}$ are updated after each mini-batch is processed ⟶ more frequent updates than batch training.
Reduced Memory Requirement: less memory-intensive ⟶ feasible for large datasets.
Balanced Convergence: It strikes a balance between the stable convergence of batch training and the faster fluctuating convergence of stochastic training.
Computational Efficiency: More computationally efficient ⟶ parallel processing and computing using GPUs.
Regularization Effect: The variability in mini-batch samples can introduce noise into the training process, which acts as a form of regularization ⟶ generalize better.
Hyperparameter: The size of the mini-batch can affect the performance and efficiency of the training process.

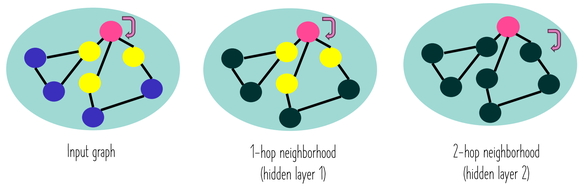
Batching and batch normalization (UDL-13)

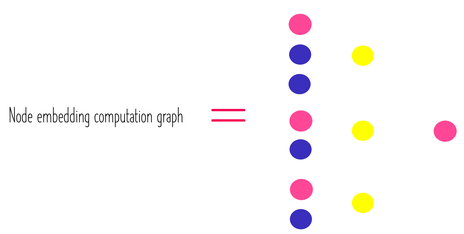
How to define the batch of training nodes for GNN training? How to sample the training nodes?

Stochastic gradient descent

For each data point:
- Calculate the gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{point}}$

receptive field of a node in a given layer

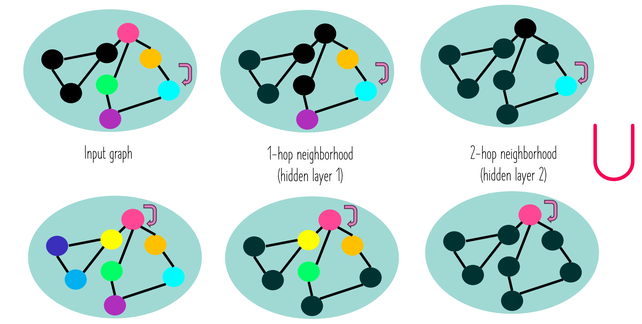


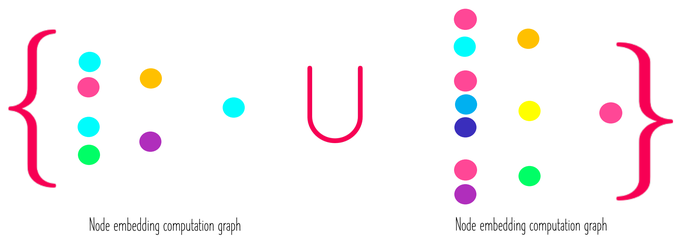


Mini-batch gradient descent — Random node sampling

For every batch:
- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{batch}}$

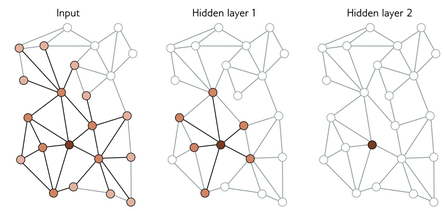
receptive field of a node in a given layer





Each node in a batch B depends on its neighbors in the previous layer.
⟶ This in turn depends on their neighbors in the layer before that (equivalent of the node receptive field).
To compute the loss at a single node at layer k, this requires the node's neighbor nodes' embeddings at layer (k-1), and the recursive ones in the downstream layers.
Mini-batch SGD uses the subgraph that forms the union of the k-hop neighbors of the nodes in B. The remaining inputs do not contribute to the node update.

What happens when the graph is dense and the GNN is deep (many layers)?

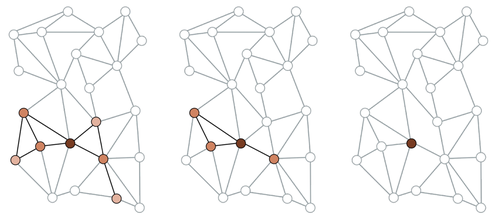


☹ Graph expansion problem: Every node may be in the receptive field of every output ⟶ this will increase the memory usage and the compute budget
☹ Costly to train and sometimes impossible (exponential time complexity to the GCN depth)!

How can we solve this better?
How to batch smartly to speed up GNN training and lower the memory usage?

Node sampling — Random neighborhood sampling

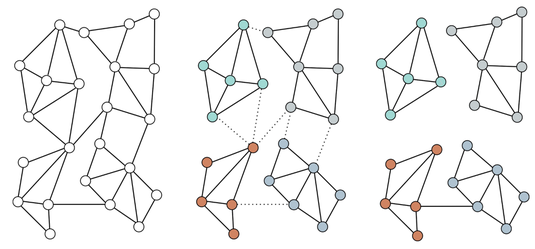
GraphSAGE (Hamilton et al., 2017):
1) Randomly define mini-batches of nodes.
2) Randomly sample a fixed number of their neighbors in the previous layer.
3) Then, randomly sample a fixed number of their neighbors in the layer before, and so on.



Will the node subgraph increase with each layer?
⟶ Yes! The graph still increases in size with each layer, but in a way that is much more controlled.
If the same batch is drawn twice, will we have the same updated embedding $\boldsymbol{h_{k+1}}$ for node $\boldsymbol{i}$?
⟶ No, since the contributing neighbors are selected randomly. So even when the same batch is drawn twice, we will have different embeddings for the node in the batch. Such randomness acts as a regularizer of network learning behavior.

Subgraph sampling — Graph partitioning

1) Cluster the graph nodes into disjoint subsets of nodes before processing. Use clustering methods to maximize the links within each cluster.
2) Treat each cluster as a batch or a random combination of them can be combined to form a batch by reinstating any links between them from the original graph.



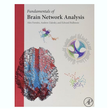
BASIRA
https://basira-lab.com/

YouTube Search "BASIRA Lab"

https://github.com/basiralab/DGL

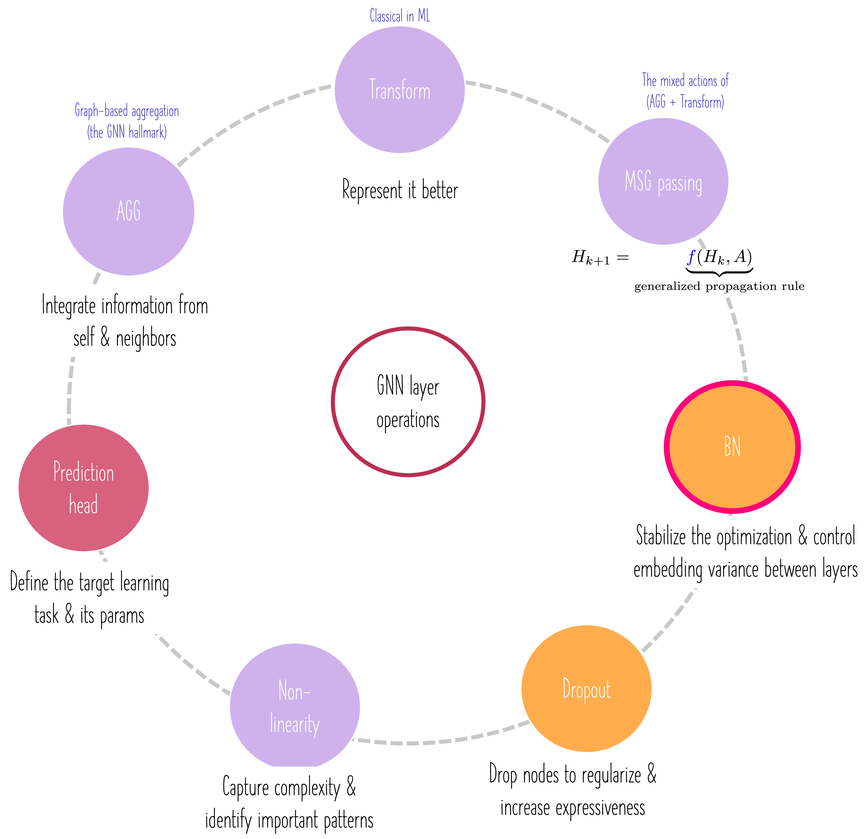
DGL 4 — Batching, Sampling & Learning Paradigms (Inductive vs Transductive)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks



Recap on GNN sampling methods

To lower the GNN compute budget, one can adopt one of the following sampling strategies:

Node sampling

Randomly select a subset of target nodes and then work back through the network adding a subset of the nodes in the receptive field at each stage.
Examples: GraphSAGE (Hamilton et al., 2017), VRGCN (Chen et al., 2017), PinSAGE (Ying et al., 2018), etc.

☺ Pros:
Computational Efficiency: Node sampling reduces the computational cost by training on a subset of nodes rather than the entire graph, making it suitable for large graphs.
Memory Efficiency: It allows for more efficient memory usage, particularly when dealing with graphs with a large number of nodes.
☹ Cons:
Redundant computations: The recursive nature of node-wise sampling brings in redundancy for calculating embeddings. If two nodes share the same neighbor, the embedding of this neighbor has to be calculated twice.

Layer sampling

Layer sampling samples the receptive field in each layer independently. This addresses the node increase in node sampling as we pass back through the graph.
Examples: FastGCN (Chen et al., 2018), adaptive sampling (Huang et al., 2018), (LADIES) layer-dependent importance sampling (Zou et al., 2019), L²-GCN (You et al., 2020), etc.

☺ Pros:
Computational Efficiency: Layer-wise sampling aims to avoid the redundant computations in node-wise sampling.
Memory Efficiency: It allows for more efficient memory usage, especially when dealing with graphs with a large number of nodes.
☹ Cons:
Local structure disruption: It may disrupt the links connecting neighboring nodes between two consecutive layers due to the random independent sampling in each layer. This may create isolated nodes and sparsifies the graph.

Subgraph sampling

Randomly draws subgraphs or divide the original graph into subgraphs. These are trained as independent data samples.
Examples: GraphSAINT (Zeng et al., 2019), ClusterGCN (Chiang et al., 2019), etc.

☺ Pros:
Computational Efficiency: Subgraph sampling significantly reduces the size of the data the GNN has to process at each iteration, making training on large graphs more feasible.
Scalability: It allows GNNs to scale to very large graphs by breaking them down into manageable pieces.
☹ Cons:
Complexity in Sampling: Designing efficient and effective subgraph sampling strategies can be complex and may require additional computational resources.

☹ Shared cons:
1) Information Loss: Sampling may result in information loss, as the model is trained on a subset of the graph instead of the entire structure.
2) Sampling Bias: The sampling process may introduce biases, especially if certain parts of the graph are consistently under-sampled.
3) Limited Global Information: Sampling might limit the model's ability to capture global graph properties, which could be crucial for certain tasks.

Batching and batch normalization (UDL-13)

Subgraph sampling

Graph partitioning

ClusterGCN (Chiang et al., 2019)

1) Sample a block of nodes that associate with a dense subgraph identified by a graph clustering algorithm, and restrict the neighborhood search within this subgraph. This maximizes the within-batch edges. A cluster defines a batch of training nodes.
2) This simple but effective strategy leads to significantly improved memory and computational efficiency while being able to achieve comparable test accuracy with previous algorithms.

Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks

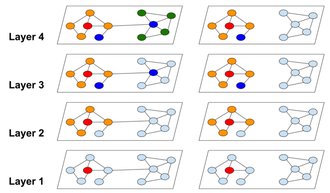


Figure 1: The neighborhood expansion difference between traditional graph convolution and our proposed cluster approach. The red node is the starting node for neighborhood nodes expansion. Traditional graph convolution suffers from exponential neighborhood expansion, while our method can avoid expensive neighborhood expansion.

Layer sampling

Idea: Sample nodes in each layer independently.
FastGCN (Chen et al., 2018):

1) Calculate a sampling probability based on the degree of each node, and sample a fixed number of nodes for each layer accordingly.
2) Only use the sampled nodes to build a much smaller sampled adjacency matrix, and thus the computation cost is reduced → the sampled nodes from two consecutive layers are not necessarily connected.

LADIES (Zou et al., 2019):

Node sampling | Layer sampling

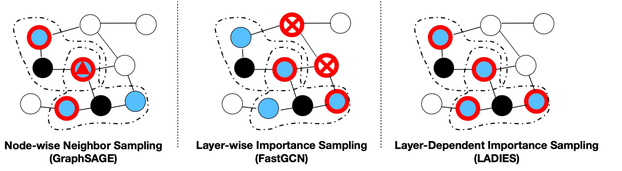


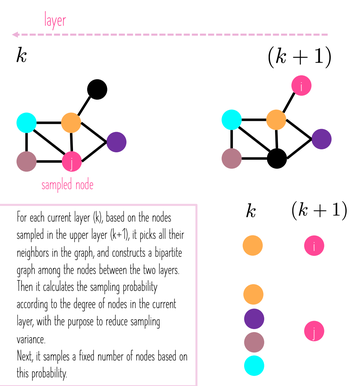
Figure 1: An illustration of the sampling process of GraphSage, FastGCN, and our proposed LADIES. Black nodes denote the nodes in the upper layer, blue nodes in the dashed circle are their neighbors, and node with the red frame is the sampled nodes. As is shown in the figure, GraphSAGE will redundantly sample a neighboring node twice, denoted by the red triangle, while FastGCN will sample nodes outside of the neighborhood. Our proposed LADIES can avoid these two problems.

Layer-Dependent Importance Sampling for Training Deep and Large Graph Convolutional Networks

Difan Zou*, Ziniu Hu*, Yewen Wang, Song Jiang, Yizhou Sun, Quanquan Gu
Department of Computer Science, UCLA, Los Angeles, CA 90095
{knowzou,bull,wyw1re804,songjiang,yzsun,qgu}@cs.ucla.edu

Abstract

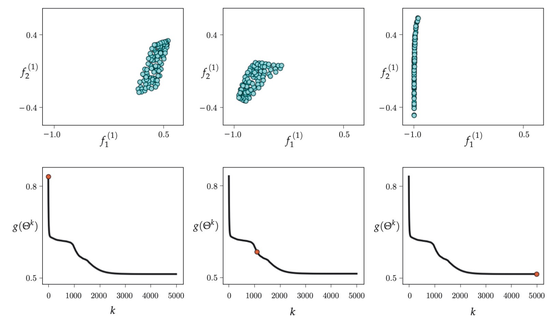
Graph convolutional networks (GCNs) have recently received wide attentions, due to their successful applications in different graph tasks and different domains. Training GCNs for a large graph, however, is still a challenge. Original full-batch GCN training requires calculating the representation of all the nodes in the graph per GCN layer, which brings in high computation and memory costs. To alleviate this issue, several sampling-based methods have been proposed to train GCNs on a subset of nodes. Among them, the node-wise neighbor-sampling method recursively samples a fixed number of neighbor nodes, and thus its computation cost suffers from exponential growing neighbor size; while the layer-wise importance-sampling method discards the neighbor-dependent constraints, and thus the nodes sampled across layer suffer from sparse connection problem. To deal with the above two problems, we propose a new effective sampling algorithm called LAyer-Dependent ImportancE Sampling (LADIES) [2]. Based on the sampled nodes in the upper layer, LADIES selects their neighborhood nodes, constructs a bipartite subgraph and computes the importance probability accordingly. Then, it samples a fixed number of nodes by the calculated probability, and recursively conducts such procedure per layer to construct the whole computation graph. We prove theoretically and experimentally, that our proposed sampling algorithm outperforms the previous sampling methods in terms of both time and memory costs. Furthermore, LADIES is shown to have better generalization accuracy than original full-batch GCN, due to its stochastic nature.



(1) For each current layer (k), based on the nodes sampled in the upper layer (k+1), it picks all their neighbors in the graph, and constructs a bipartite graph among the nodes between the two layers.
(2) Then it calculates the sampling probability according to the degree of nodes in the current layer, with the purpose to reduce sampling variance.
(3) Next, it samples a fixed number of nodes based on this probability.

Batch normalization (BN)

Aim: Stabilize the GNN training.



Batch normalization or *BatchNorm* shifts and rescales each activation h so that its mean and variance across the batch $\mathcal{B}$ become values which are learned during training. First, the empirical mean m_h and standard deviation s_h are computed:

Compute the mean and variance over |B| embeddings in a batch B.

$$\begin{aligned} m_h &= \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} h_i \\ s_h &= \sqrt{\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(h_i - m_h)^2}. \end{aligned} \tag{11.7}$$

Normalize the node embeddings using the computed mean and variance

$$h_i \leftarrow \frac{h_i - m_h}{s_h + \epsilon} \qquad \forall i \in \mathcal{B},$$

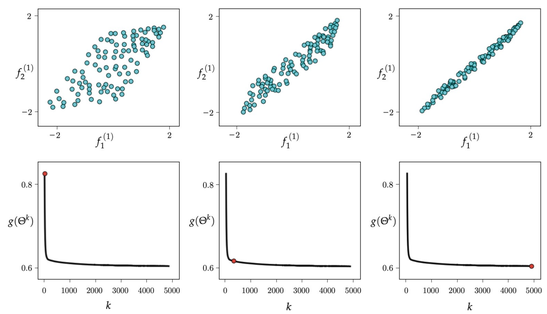
$$h_i \leftarrow \gamma h_i + \delta \qquad \forall i \in \mathcal{B},$$

Benefits of batch normalization

Higher learning rates: BN makes the loss surface and its gradient change more smoothly (i.e., reduces shattered gradients)
→ one can use higher learning rates as the surface is more predictable and smoother.

Regularization: BN injects noise to the training process because the normalization depends on the statistics of the entire batch.
→ injecting noise makes models generalize better.
For instance, the activations of a given training sample will be normalized by an amount that depends on the members of the batch, and so will be processed slightly differently depending on the batch elements.

Stable forward propagation: BN allows to control the variance of the learned embeddings between consecutive layers, which stabilizes the network transformation functions between neighboring layers.
→ the network can easily control its own effective learning path.



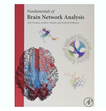
BASIRA https://basira-lab.com/

YouTube Search "BASIRA Lab"

https://github.com/basiralab/DGL

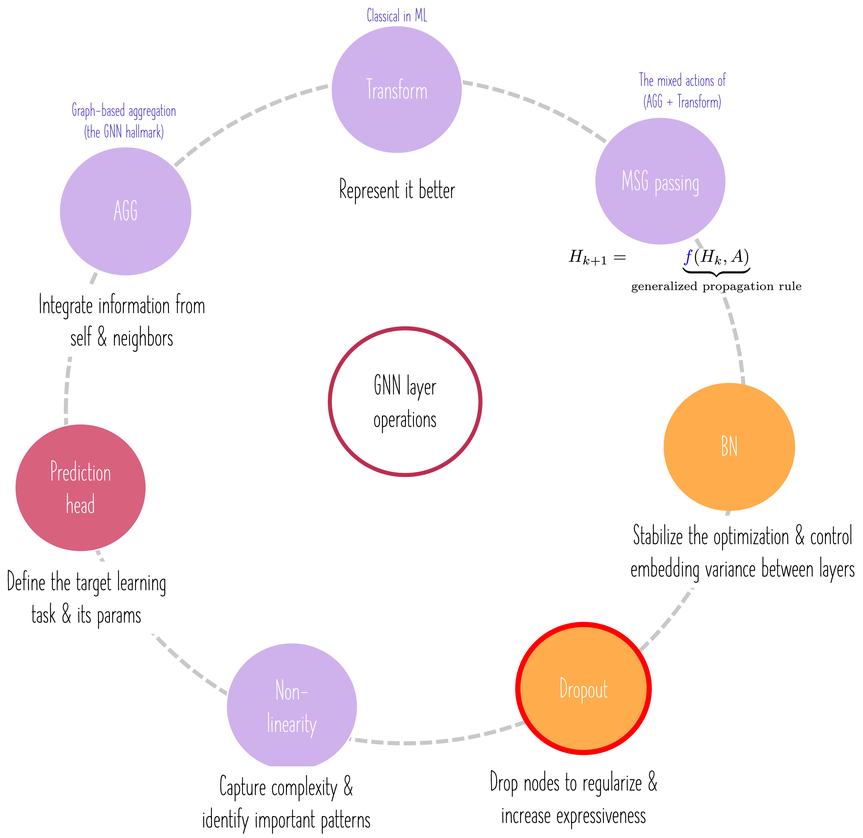
DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



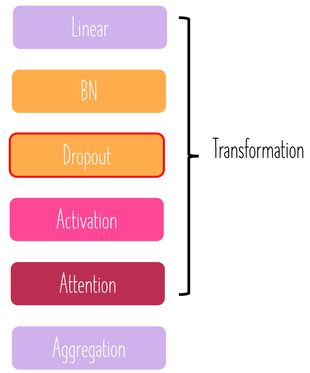

https://github.com/udlbook/

GNN layer operations & building blocks



Dropout layer (following BN)

A single GNN layer can take the following form:



One can include different deep learning modules depending on the downstream task.
Example: add MLP layers before and/or after a GNN layer.
The design of a general GNN layer is an ongoing area of research.

Conventional Dropout

Aim: regularize a neural network to avoid overfitting. Dropout is applied right after a BN layer.
During training: randomly set neurons to zero (drop out) with a probability p in a particular layer.
During testing: use all neurons for computation.

Dropout →

Dropout for GNNs

In GNN, Dropout follows up the BN layer.
In the absence of a BN layer, Dropout is applied to the linear layer directly in the message function.
→ Randomly drops training nodes.

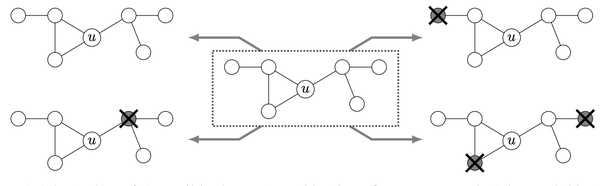


Figure 1: Illustration of 4 possible dropout combinations from an example 2-hop neighborhood around u: a 0-dropout, two different 1-dropouts and a 2-dropout. DropGNN (Papp et al., 2021).

DropGNN (2021)

DropGNN: Random Dropouts Increase the Expressiveness of Graph Neural Networks

Pál András Papp ETH Zurich apapp@ethz.ch
Karolis Martinkus ETH Zurich martinkus@ethz.ch
Lukas Faber ETH Zurich lfaber@ethz.ch
Roger Wattenhofer ETH Zurich wattenhofer@ethz.ch

In each of these runs, we remove ("drop out") each node in the graph with a small probability p. As such, the different runs of an episode will allow us to not only observe the actual extended neighborhood of a node for some number of layers d, but rather to observe various slightly perturbed versions of this d-hop neighborhood. We emphasize that this notion of dropouts is very different from the popular dropout regularization method; in particular, DropGNNs remove nodes during *both training and testing*, since their goal is to observe a similar distribution of dropout patterns during training and testing.

Our contributions. We begin by showing several example graphs that are not distinguishable in the regular GNN setting but can be easily separated by DropGNNs. We then analyze the theoretical properties of DropGNNs in detail. We first show that executing $\widetilde{O}(\gamma)$ different runs is often already sufficient to ensure that we observe a reasonable distribution of dropouts in a neighborhood of size γ. We then discuss the theoretical capabilities and limitations of DropGNNs in general, as well as the limits of the dropout approach when combined with specific aggregation methods.

We validate our theoretical findings on established problems that are impossible to solve for standard GNNs. We find that DropGNNs clearly outperform the competition on these datasets. We further show that DropGNNs have a competitive performance on several established graph benchmarks, and they provide particularly impressive results in applications where the graph structure is really a crucial factor.

GNN expressive power or expressiveness

The ability to model and capture the diverse structural patterns and features in a graph. It is also the ability to distinguish between different graph structures and accurately map graph inputs to outputs.

Node embedding computation graph

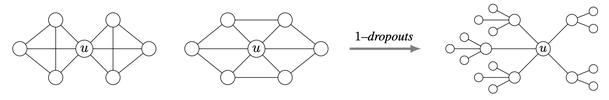
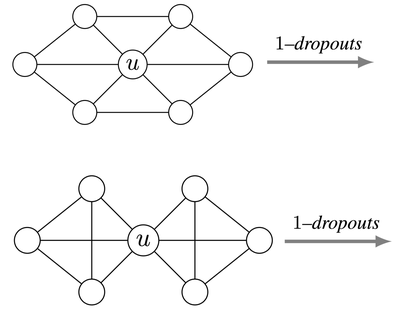
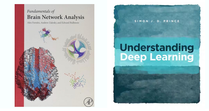


Figure 3: Example of two graphs not separable by 1-dropouts (left side). In both of the graphs, for any of the 1-dropouts, u observes the same tree structure for $d = 2$, shown on the right side.

u embedding computation graph



Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



Inductive vs transductive GNN training and testing (UDL-13)

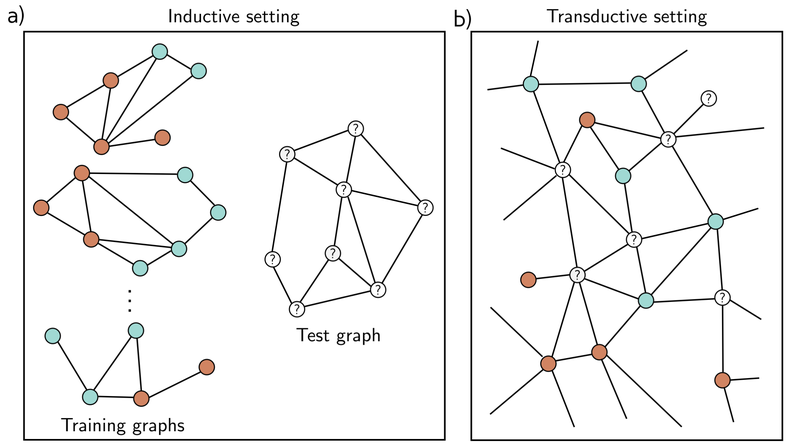
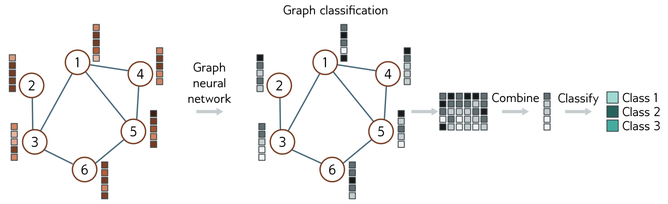


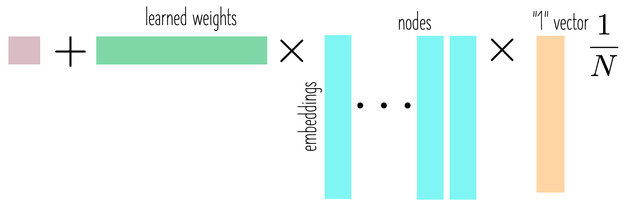
Figure 13.8 Inductive vs. transductive problems. a) Node classification task in inductive setting. We are given a set of I training graphs, where the node labels (orange and cyan colors) are known. After training, we are given a test graph and must assign labels to each node. b) Node classification in transductive setting. There is one large graph in which some of the nodes have labels (orange and cyan colors) and others are unknown. We train the model to predict the known labels correctly and then examine the predictions at the unknown nodes.

Inductive learning	Transductive learning
☺ We learn the rule that maps the graph (or its elements) to a target output, then apply this rule to new data.	☹ It does not produce a rule, but merely a labeling for all of the unknown outputs.
☹ Cannot access the original features of the testing samples during the training stage.	☺ Considers both the labeled and unlabeled data at the same time. ☺ It has the advantage that it can use patterns in the unlabeled data to help make its decisions.
☺ No need to store the testing nodes and their original features in memory.	☹ High memory cost as it jointly stores information for training and testing node. ☹ Higher computational time.
☺ Test the trained model on unseen nodes (new ones).	☹ When extra unlabeled data (new testing samples) are added, the model should be retrained from scratch.

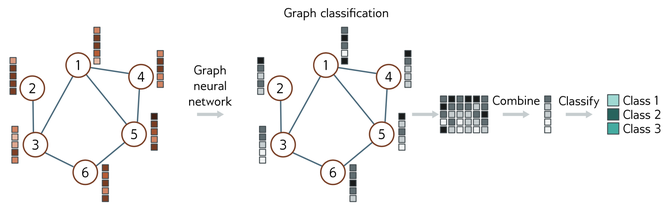
Inductive ~~node~~ graph classification



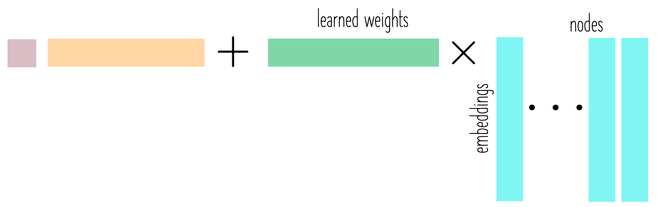
$$y = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{H}_K \mathbf{1}/N\right]$$



Transductive node classification



$$y = \mathbf{sig}[\beta_K \mathbf{1}^T + \omega_K \mathbf{H}_K]$$



$\mathbf{sig}(\cdot)$ applies the sigmoid function independently to every element of the row vector input.

Question for reflection: How to batch nodes in a transductive learning paradigm?

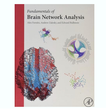
Recall box: summarize what you remember from this lecture below

YOU CAN TAKE NOTES
ON THE FOLLOWING SLIDES

Let us reflect and self-quiz!

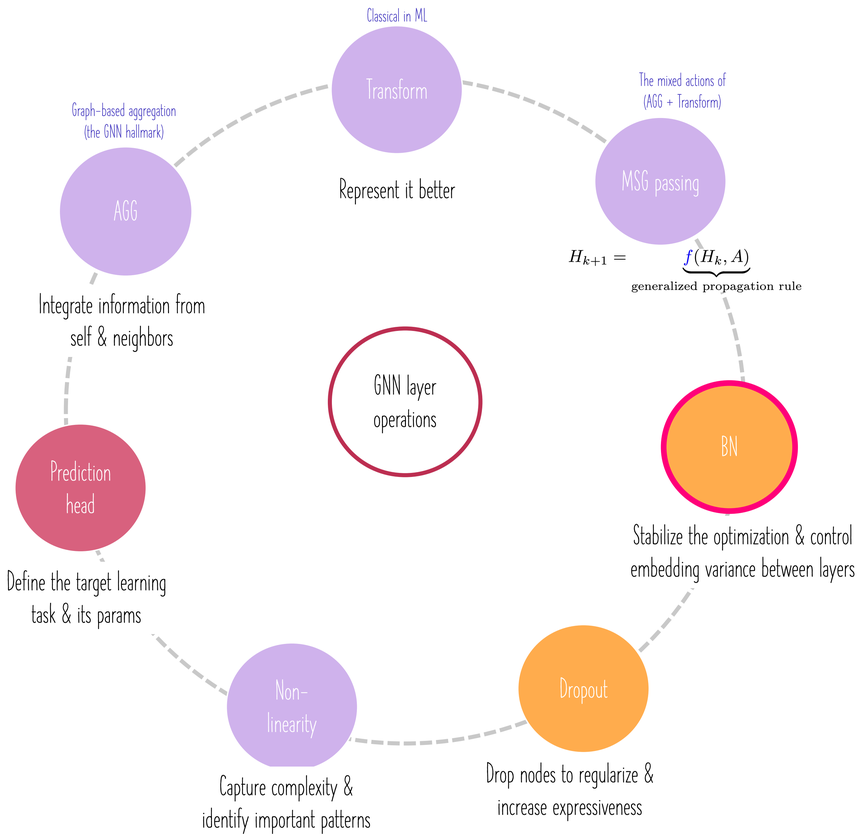
DGL 4 — Batching, Sampling & Learning Paradigms (Inductive vs Transductive)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks



Batching and batch normalization (UDL-13)

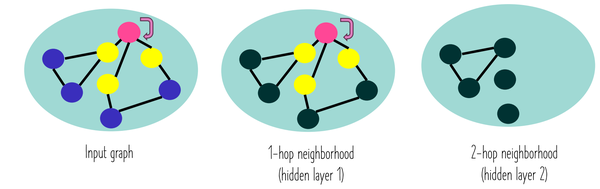
How to define the batch of training nodes for GNN training? How to sample the training nodes?

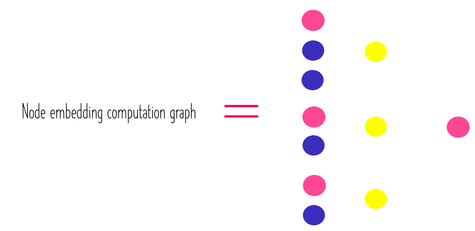
Stochastic gradient descent

For each data point:

- Calculate the gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{point}}$

receptive field of a node in a given layer



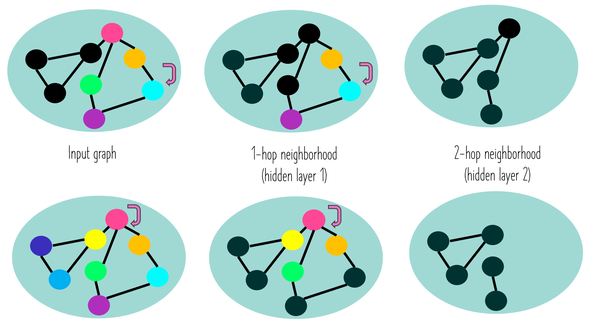


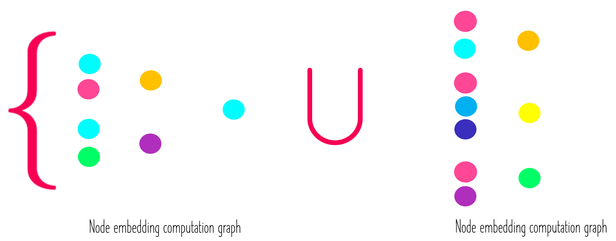
Mini-batch gradient descent Random node sampling

For every batch:

- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{batch}}$

receptive field of a node in a given layer



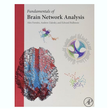


Each node in a batch B depends on its neighbors in the previous layer.
→ This in turn depends on their neighbors in the layer before that (equivalent of the node receptive field).
To compute the loss at a single node at layer k, this requires the node's neighbor nodes' embeddings at layer (k-1), and the recursive ones in the downstream layers.
Mini-batch SGD uses the subgraph that forms the union of the k-hop neighbors of the nodes in B. The remaining inputs do not contribute to the node update.

Personal reflections and notes

DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks

Personal reflections and notes

Batching and batch normalization (UDL-13)

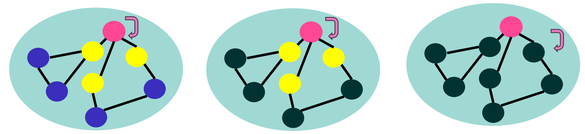
How to define the batch of training nodes for GNN training? How to sample the training nodes?

Stochastic gradient descent

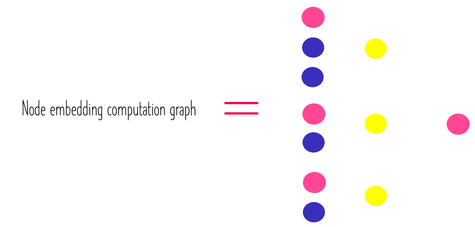
For each data point:

- Calculate the gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{point}}$

receptive field of a node in a given layer



Input graph | 1-hop neighborhood (hidden layer 1) | 2-hop neighborhood (hidden layer 2)



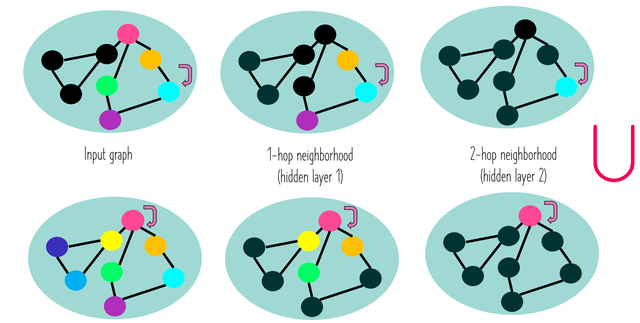
Node embedding computation graph

Mini-batch gradient descent — Random node sampling

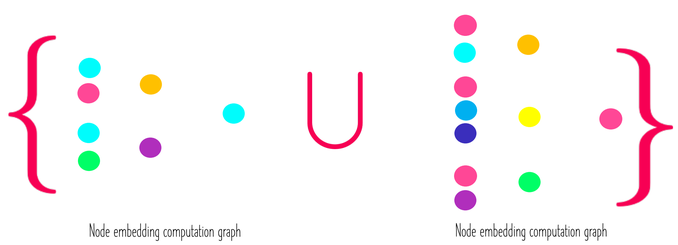
For every batch:

- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{batch}}$

receptive field of a node in a given layer



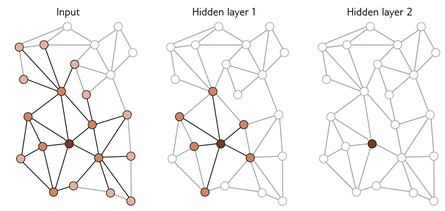
Input graph | 1-hop neighborhood (hidden layer 1) | 2-hop neighborhood (hidden layer 2)



Node embedding computation graph | Node embedding computation graph

Each node in a batch B depends on its neighbors in the previous layer.
→ This in turn depends on their neighbors in the layer before that (equivalent of the node receptive field).
To compute the loss at a single node at layer k, this requires the node's neighbor nodes' embeddings at layer (k-1), and the recursive ones in the downstream layers.
Mini-batch SGD uses the subgraph that forms the union of the k-hop neighbors of the nodes in B. The remaining inputs do not contribute to the node update.

What happens when the graph is dense and the GNN is deep (many layers)?



Input | Hidden layer 1 | Hidden layer 2

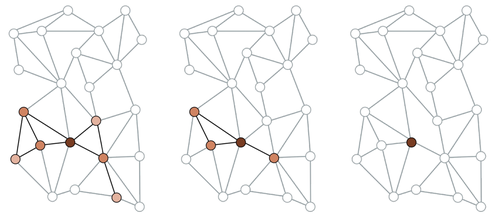
☹ Graph expansion problem: Every node may be in the receptive field of every output → this will increase the memory usage and the compute budget
☹ Costly to train and sometimes impossible (exponential time complexity to the GCN depth)!

How can we solve this better?
How to batch smartly to speed up GNN training and lower the memory usage?

Node sampling — Random neighborhood sampling

GraphSAGE (Hamilton et al., 2017):

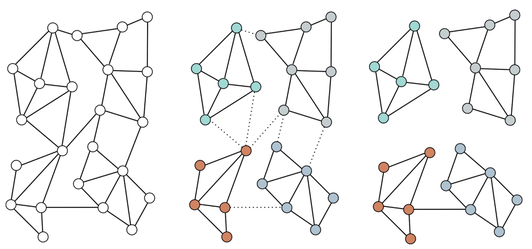
1) Randomly define mini-batches of nodes.
2) Randomly sample a fixed number of their neighbors in the previous layer.
3) Then, randomly sample a fixed number of their neighbors in the layer before, and so on.



Will the node subgraph increase with each layer?
→ Yes! The graph still increases in size with each layer, but in a way that is much more controlled.
If the same batch is drawn twice, will we have the same updated embedding $\boldsymbol{h_{k+1}}$ for node $\boldsymbol{i}$?
→ No, since the contributing neighbors are selected randomly. So even when the same batch is drawn twice, we will have different embeddings for the node in the batch. Such randomness acts as a regularizer of network learning behavior.

Subgraph sampling — Graph partitioning

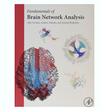
1) Cluster the graph nodes into disjoint subsets of nodes before processing. Use clustering methods to maximize the links within each cluster.
2) Treat each cluster as a batch or a random combination of them can be combined to form a batch by reinstating any links between them from the original graph.



Original graph | 4 clusters (batches) | 3 batches

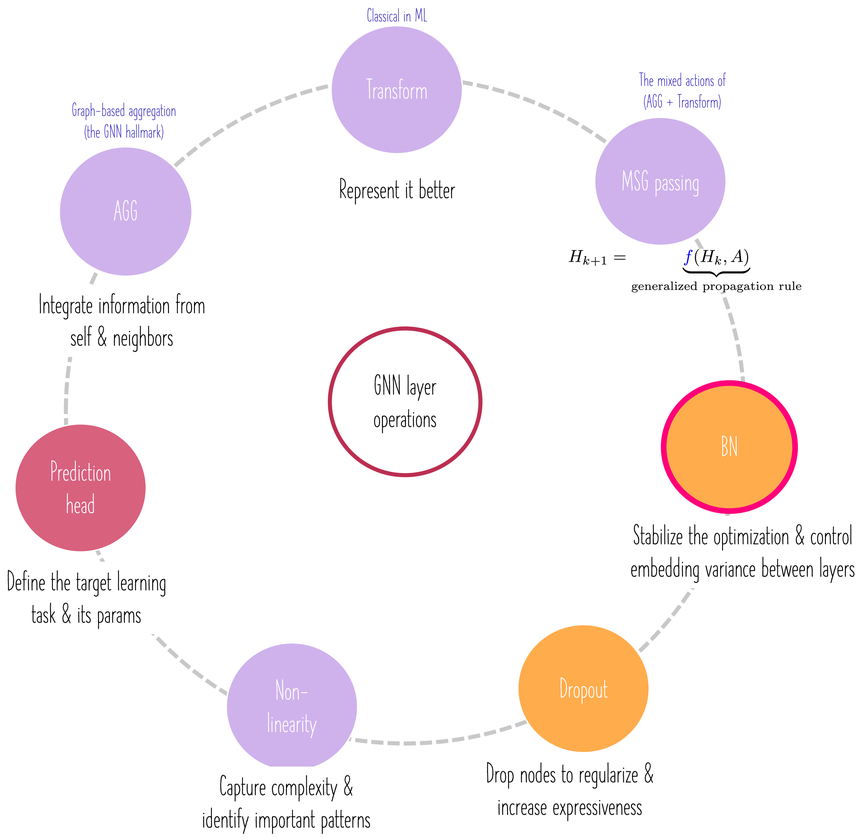
DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks



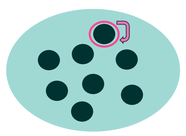
Reminder

To train a model by optimizing the loss objective, one can adopt one of the following 3 training strategies:

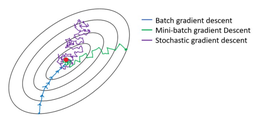
1) Stochastic gradient descent

For each data point:

- Calculate the gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{point}}$



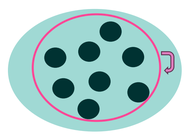
Individual Data Points: In single-point training (or stochastic training), each update is based on the gradient calculated from a single randomly chosen data point.
Frequent Updates: Parameters $\boldsymbol{w}$ are updated frequently after every single point ⟶ $\boldsymbol{w}$ are less stable and can exhibit more variability.
Noisy Updates: The updates can be noisy, leading to more erratic convergence behavior.
Computational Efficiency: It is computationally inefficient especially when dealing with large datasets.



2) Batch gradient descent

For the whole dataset:

- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{dataset}}$

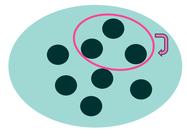


Full Dataset Usage: In batch training, the entire dataset is used for each training iteration ⟶ the model parameters $\boldsymbol{w}$ are updated only once per epoch (an epoch is a complete pass through the entire training dataset).
Memory Requirements: The entire dataset is loaded into memory ⟶ very memory-intensive, and impractical for large datasets.
Stability: It provides stable error gradients and convergence, as the gradient calculation is based on the entire dataset, leading to a consistent update direction.
Computational Efficiency: Computationally inefficient due to the need to process the entire dataset before making a single update.
Risk of Overfitting: since the model is repeatedly exposed to the entire dataset in the same order (no randomness for regularization).

3) Mini-batch gradient descent

For every batch:

- Calculate the average gradient
- Update $\boldsymbol{w_{t+1} = w_t - \eta \nabla L_{batch}}$



Subset of Dataset: Mini-batch training involves dividing the dataset into smaller subsets or "mini-batches." ⟶ The parameters $\boldsymbol{w}$ are updated after each mini-batch is processed ⟶ more frequent updates than batch training.
Reduced Memory Requirement: less memory-intensive ⟶ feasible for large datasets.
Balanced Convergence: It strikes a balance between the stable convergence of batch training and the faster fluctuating convergence of stochastic training.
Computational Efficiency: More computationally efficient ⟶ parallel processing and computing using GPUs.
Regularization Effect: The variability in mini-batch samples can introduce noise into the training process, which acts as a form of regularization ⟶ generalize better.
Hyperparameter: The size of the mini-batch can affect the performance and efficiency of the training process.

Personal reflections and notes

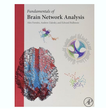
BASIRA
https://basira-lab.com/

YouTube Search "BASIRA Lab"

https://github.com/basiralab/DGL

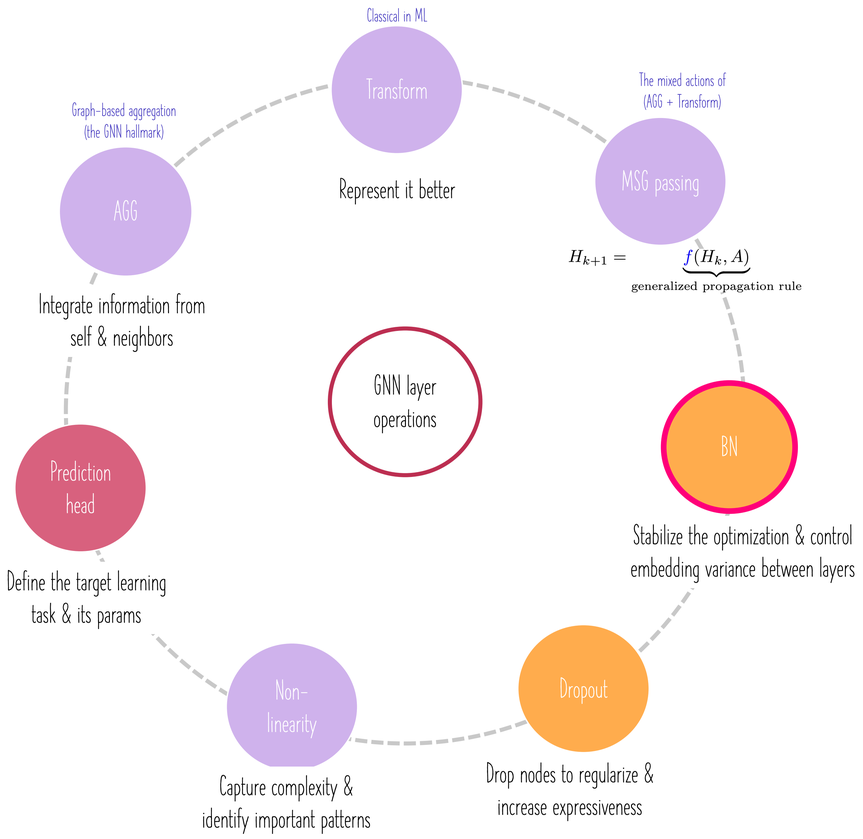
DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks



Batching and batch normalization (UDL-13)

Subgraph sampling

Graph partitioning

ClusterGCN (Chiang et al., 2019)

1) Sample a block of nodes that associate with a dense subgraph identified by a graph clustering algorithm, and restrict the neighborhood search within this subgraph. This maximizes the within-batch edges. A cluster defines a batch of training nodes.
2) This simple but effective strategy leads to significantly improved memory and computational efficiency while being able to achieve comparable test accuracy with previous algorithms.

Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks

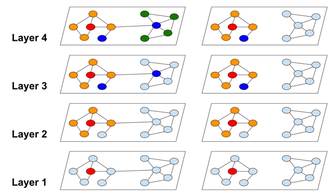


Figure 1: The neighborhood expansion difference between traditional graph convolution and our proposed cluster approach. The red node is the starting node for neighborhood nodes expansion. Traditional graph convolution suffers from exponential neighborhood expansion, while our method can avoid expensive neighborhood expansion.

Layer sampling

Idea: Sample nodes in each layer independently.

FastGCN (Chen et al., 2018):

1) Calculate a sampling probability based on the degree of each node, and sample a fixed number of nodes for each layer accordingly.
2) Only use the sampled nodes to build a much smaller sampled adjacency matrix, and thus the computation cost is reduced → the sampled nodes from two consecutive layers are not necessarily connected.

LADIES (Zou et al., 2019):

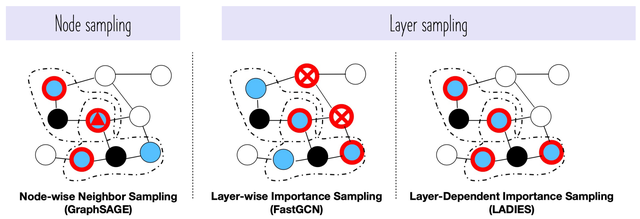


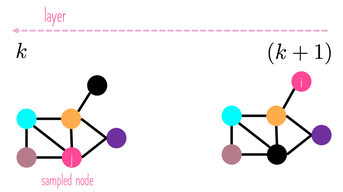
Figure 1: An illustration of the sampling process of GraphSage, FastGCN, and our proposed LADIES. Black nodes denote the nodes in the upper layer, blue nodes in the dashed circle are their neighbors, and node with the red frame is the sampled nodes. As is shown in the figure, GraphSAGE will redundantly sample a neighboring node twice, denoted by the red triangle, while FastGCN will sample nodes outside of the neighborhood. Our proposed LADIES can avoid these two problems.

Layer-Dependent Importance Sampling for Training Deep and Large Graph Convolutional Networks

Difan Zou*, Ziniu Hu*, Yewen Wang, Song Jiang, Yizhou Sun, Quanquan Gu
Department of Computer Science, UCLA, Los Angeles, CA 90095
{knowzou,bill,wyw10804,songjiang,yzsun,qgu}@cs.ucla.edu

Abstract

Graph convolutional networks (GCNs) have recently received wide attentions, due to their successful applications in different graph tasks and different domains. Training GCNs for a large graph, however, is still a challenge. Original full-batch GCN training requires calculating the representation of all the nodes in the graph per GCN layer, which brings in high computation and memory costs. To alleviate this issue, several sampling-based methods have been proposed to train GCNs on a subset of nodes. Among them, the node-wise neighbor-sampling method recursively samples a fixed number of neighbor nodes, and thus its computation cost suffers from exponential growing neighbor size; while the layer-wise importance-sampling method discards the neighbor-dependent constraints, and thus the nodes sampled across layer suffer from sparse connection problem. To deal with the above two problems, we propose a new effective sampling algorithm called LAyer-Dependent ImportancE Sampling (LADIES) *. Based on the sampled nodes in the upper layer, LADIES selects their neighborhood nodes, constructs a bipartite subgraph and computes the importance probability accordingly. Then, it samples a fixed number of nodes by the calculated probability, and recursively conducts such procedure per layer to construct the whole computation graph. We prove theoretically and experimentally, that our proposed sampling algorithm outperforms the previous sampling methods in terms of both time and memory costs. Furthermore, LADIES is shown to have better generalization accuracy than original full-batch GCN, due to its stochastic nature.



(1) For each current layer (k), based on the nodes sampled in the upper layer (k+1), it picks all their neighbors in the graph, and constructs a bipartite graph among the nodes between the two layers.
(2) Then it calculates the sampling probability according to the degree of nodes in the current layer, with the purpose to reduce sampling variance.
(3) Next, it samples a fixed number of nodes based on this probability.

Personal reflections and notes

BASIRA

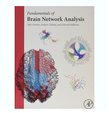
https://basira-lab.com/

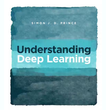
YouTube Search "BASIRA Lab"

https://github.com/basiralab/DGL

DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.





https://github.com/udlbook/

GNN layer operations & building blocks

Personal reflections and notes

Batching and batch normalization (UDL-13)

Subgraph sampling

Graph partitioning

ClusterGCN (Chiang et al., 2019)

1) Sample a block of nodes that associate with a dense subgraph identified by a graph clustering algorithm, and restrict the neighborhood search within this subgraph. This maximizes the within-batch edges. A cluster defines a batch of training nodes.
2) This simple but effective strategy leads to significantly improved memory and computational efficiency while being able to achieve comparable test accuracy with previous algorithms.

Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks

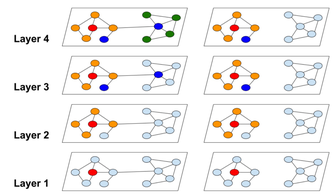


Figure 1: The neighborhood expansion difference between traditional graph convolution and our proposed cluster approach. The red node is the starting node for neighborhood nodes expansion. Traditional graph convolution suffers from exponential neighborhood expansion, while our method can avoid expensive neighborhood expansion.

Layer sampling

Idea: Sample nodes in each layer independently.

FastGCN (Chen et al., 2018):

1) Calculate a sampling probability based on the degree of each node, and sample a fixed number of nodes for each layer accordingly.
2) Only use the sampled nodes to build a much smaller sampled adjacency matrix, and thus the computation cost is reduced → the sampled nodes from two consecutive layers are not necessarily connected.

LADIES (Zou et al., 2019):

Node sampling | Layer sampling

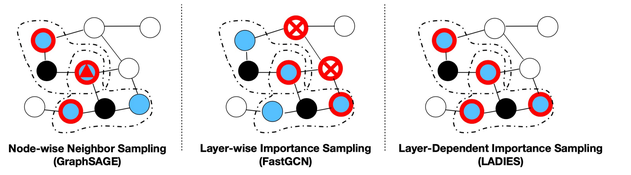


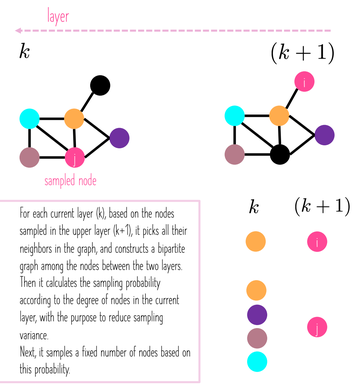
Figure 1: An illustration of the sampling process of GraphSage, FastGCN, and our proposed LADIES. Black nodes denote the nodes in the upper layer, blue nodes in the dashed circle are their neighbors, and node with the red frame is the sampled nodes. As is shown in the figure, GraphSAGE will redundantly sample a neighboring node twice, denoted by the red triangle, while FastGCN will sample nodes outside of the neighborhood. Our proposed LADIES can avoid these two problems.

Layer-Dependent Importance Sampling for Training Deep and Large Graph Convolutional Networks

Difan Zou*, Ziniu Hu*, Yewen Wang, Song Jiang, Yizhou Sun, Quanquan Gu
Department of Computer Science, UCLA, Los Angeles, CA 90095
{knowzou,bull,wyw10804,songjiang,yzsun,qgu}@cs.ucla.edu

Abstract

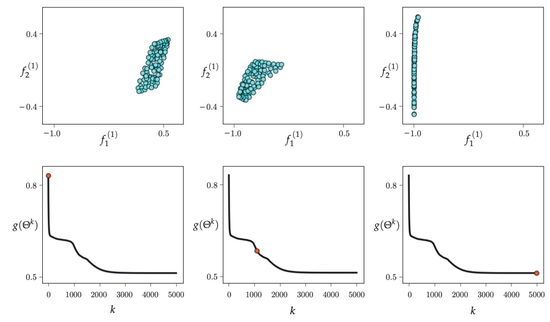
Graph convolutional networks (GCNs) have recently received wide attentions, due to their successful applications in different graph tasks and different domains. Training GCNs for a large graph, however, is still a challenge. Original full-batch GCN training requires calculating the representation of all the nodes in the graph per GCN layer, which brings in high computation and memory costs. To alleviate this issue, several sampling-based methods have been proposed to train GCNs on a subset of nodes. Among them, the node-wise neighbor-sampling method recursively samples a fixed number of neighbor nodes, and thus its computation cost suffers from exponential growing neighbor size; while the layer-wise importance-sampling method discards the neighbor-dependent constraints, and thus the nodes sampled across layer suffer from sparse connection problem. To deal with the above two problems, we propose a new effective sampling algorithm called Layer-Dependent Importance Sampling (LADIES). Based on the sampled nodes in the upper layer, LADIES selects their neighborhood nodes, constructs a bipartite subgraph and computes the importance probability accordingly. Then, it samples a fixed number of nodes by the calculated probability, and recursively conducts such procedure per layer to construct the whole computation graph. We prove theoretically and experimentally, that our proposed sampling algorithm outperforms the previous sampling methods in terms of both time and memory costs. Furthermore, LADIES is shown to have better generalization accuracy than original full-batch GCN, due to its stochastic nature.



(1) For each current layer (k), based on the nodes sampled in the upper layer (k+1), it picks all their neighbors in the graph, and constructs a bipartite graph among the nodes between the two layers.
(2) Then it calculates the sampling probability according to the degree of nodes in the current layer, with the purpose to reduce sampling variance.
(3) Next, it samples a fixed number of nodes based on this probability.

Batch normalization (BN)

Aim: Stabilize the GNN training.



Batch normalization or *BatchNorm* shifts and rescales each activation h so that its mean and variance across the batch $\mathcal{B}$ become values which are learned during training. First, the empirical mean m_h and standard deviation s_h are computed:

Compute the mean and variance over |B| embeddings in a batch B.

$$\begin{aligned} m_h &= \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} h_i \\ s_h &= \sqrt{\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(h_i - m_h)^2}. \end{aligned} \tag{11.7}$$

Normalize the node embeddings using the computed mean and variance

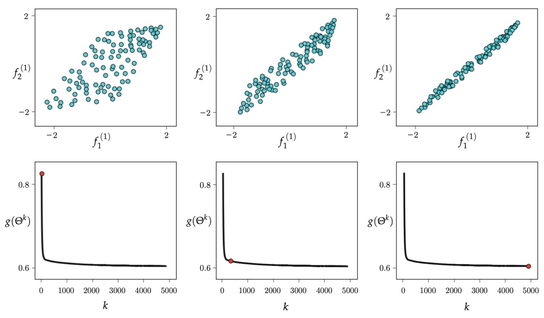
$$\begin{aligned} h_i &\leftarrow \frac{h_i - m_h}{s_h + \epsilon} && \forall i \in \mathcal{B}, \\ h_i &\leftarrow \gamma h_i + \delta && \forall i \in \mathcal{B}, \end{aligned}$$

Benefits of batch normalization

Higher learning rates: BN makes the loss surface and its gradient change more smoothly (i.e., reduces shattered gradients)
→ one can use higher learning rates as the surface is more predictable and smoother.

Regularization: BN injects noise to the training process because the normalization depends on the statistics of the entire batch.
→ injecting noise makes models generalize better.
For instance, the activations of a given training sample will be normalized by an amount that depends on the members of the batch, and so will be processed slightly differently depending on the batch elements.

Stable forward propagation: BN allows to control the variance of the learned embeddings between consecutive layers, which stabilizes the network transformation functions between neighboring layers.
→ the network can easily control its own effective learning path.



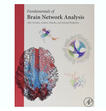
BASIRA
https://basira-lab.com/

YouTube Search "BASIRA Lab"

https://github.com/basiralab/DGL

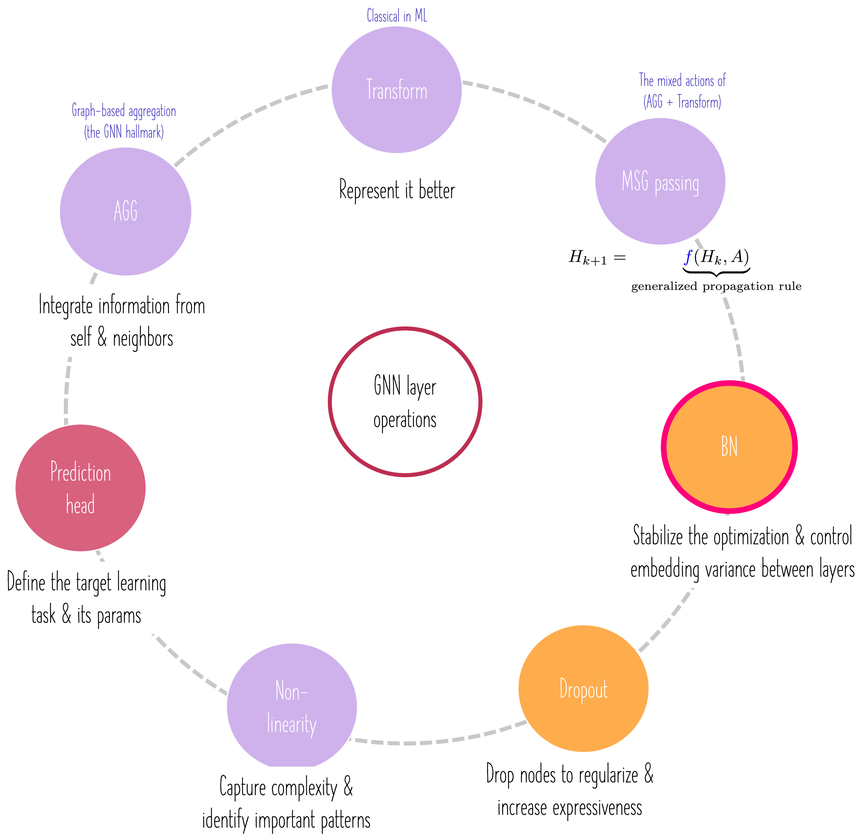
DGL 4
BATCHING, SAMPLING & LEARNING PARADIGMS
(INDUCTIVE VS TRANSDUCTIVE)

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

GNN layer operations & building blocks



Recap on GNN sampling methods

To lower the GNN compute budget, one can adopt one of the following sampling strategies:

Node sampling

Randomly select a subset of target nodes and then work back through the network adding a subset of the nodes in the receptive field at each stage.
Examples: GraphSAGE (Hamilton et al., 2017), VRGCN (Chen et al., 2017), PinSAGE (Ying et al., 2018), etc.

☺ Pros:
Computational Efficiency: Node sampling reduces the computational cost by training on a subset of nodes rather than the entire graph, making it suitable for large graphs.
Memory Efficiency: It allows for more efficient memory usage, particularly when dealing with graphs with a large number of nodes.
☹ Cons:
Redundant computations: The recursive nature of node-wise sampling brings in redundancy for calculating embeddings. If two nodes share the same neighbor, the embedding of this neighbor has to be calculated twice.

Layer sampling

Layer sampling samples the receptive field in each layer independently. This addresses the node increase in node sampling as we pass back through the graph.
Examples: FastGCN (Chen et al., 2018), adaptive sampling (Huang et al., 2018), (LADIES) layer-dependent importance sampling (Zou et al., 2019), L^2-GCN (You et al., 2020), etc.

☺ Pros:
Computational Efficiency: Layer-wise sampling aims to avoid the redundant computations in node-wise sampling.
Memory Efficiency: It allows for more efficient memory usage, especially when dealing with graphs with a large number of nodes.
☹ Cons:
Local structure disruption: It may disrupt the links connecting neighboring nodes between two consecutive layers due to the random independent sampling in each layer. This may create isolated nodes and sparsifies the graph.

Subgraph sampling

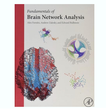
Randomly draws subgraphs or divide the original graph into subgraphs. These are trained as independent data samples.
Examples: GraphSAINT (Zeng et al., 2019), ClusterGCN (Chiang et al., 2019), etc.

☺ Pros:
Computational Efficiency: Subgraph sampling significantly reduces the size of the data the GNN has to process at each iteration, making training on large graphs more feasible.
Scalability: It allows GNNs to scale to very large graphs by breaking them down into manageable pieces.
☹ Cons:
Complexity in Sampling: Designing efficient and effective subgraph sampling strategies can be complex and may require additional computational resources.

☹ Shared cons:
1) Information Loss: Sampling may result in information loss, as the model is trained on a subset of the graph instead of the entire structure.
2) Sampling Bias: The sampling process may introduce biases, especially if certain parts of the graph are consistently under-sampled.
3) Limited Global Information: Sampling might limit the model's ability to capture global graph properties, which could be crucial for certain tasks.

Personal reflections and notes

Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



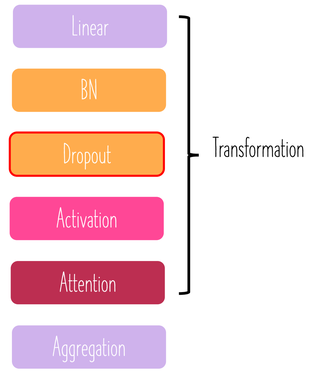
https://github.com/udlbook/

GNN layer operations & building blocks

Personal reflections and notes

Dropout layer (following BN)

A single GNN layer can take the following form:



One can include different deep learning modules depending on the downstream task.
Example: add MLP layers before and/or after a GNN layer.
The design of a general GNN layer is an ongoing area of research.

Conventional Dropout

Aim: regularize a neural network to avoid overfitting. Dropout is applied right after a BN layer.
During training: randomly set neurons to zero (drop out) with a probability p in a particular layer.
During testing: use all neurons for computation.

Dropout for GNNs

In GNN, Dropout follows up the BN layer.
In the absence of a BN layer, Dropout is applied to the linear layer directly in the message function.
→ Randomly drops training nodes.

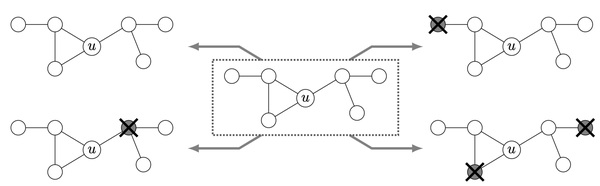


Figure 1: Illustration of 4 possible dropout combinations from an example 2-hop neighborhood around u: a 0-dropout, two different 1-dropouts and a 2-dropout. DropGNN (Papp et al., 2021).

DropGNN (2021)

DropGNN: Random Dropouts Increase the Expressiveness of Graph Neural Networks

Pál András Papp ETH Zurich apapp@ethz.ch
Karolis Martinkus ETH Zurich martinkus@ethz.ch
Lukas Faber ETH Zurich lfaber@ethz.ch
Roger Wattenhofer ETH Zurich wattenhofer@ethz.ch

In each of these runs, we remove ("drop out") each node in the graph with a small probability p. As such, the different runs of an episode will allow us to not only observe the actual extended neighborhood of a node for some number of layers d, but rather to observe various slightly perturbed versions of this d-hop neighborhood. We emphasize that this notion of dropouts is very different from the popular dropout regularization method; in particular, DropGNNs remove nodes during *both training and testing*, since their goal is to observe a similar distribution of dropout patterns during training and testing.

Our contributions. We begin by showing several example graphs that are not distinguishable in the regular GNN setting but can be easily separated by DropGNNs. We then analyze the theoretical properties of DropGNNs in detail. We first show that executing $\widetilde{O}(\gamma)$ different runs is often already sufficient to ensure that we observe a reasonable distribution of dropouts in a neighborhood of size γ. We then discuss the theoretical capabilities and limitations of DropGNNs in general, as well as the limits of the dropout approach when combined with specific aggregation methods.

We validate our theoretical findings on established problems that are impossible to solve for standard GNNs. We find that DropGNNs clearly outperform the competition on these datasets. We further show that DropGNNs have a competitive performance on several established graph benchmarks, and they provide particularly impressive results in applications where the graph structure is really a crucial factor.

GNN expressive power or expressiveness

The ability to model and capture the diverse structural patterns and features in a graph. It is also the ability to distinguish between different graph structures and accurately map graph inputs to outputs.

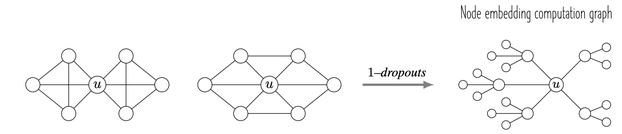
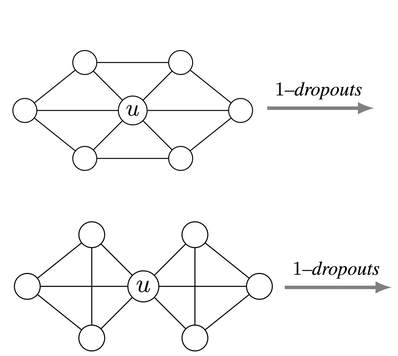
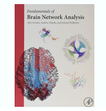


Figure 3: Example of two graphs not separable by 1-dropouts (left side). In both of the graphs, for any of the 1-dropouts, u observes the same tree structure for $d = 2$, shown on the right side.



Special thanks to Simon Prince, Alex Fornito, Andrew Zalesky, Edward Bullmore, Jure Leskovec and all those who shared their passion about graphs and deep learning.



https://github.com/udlbook/

Inductive vs transductive GNN training and testing (UDL-13)

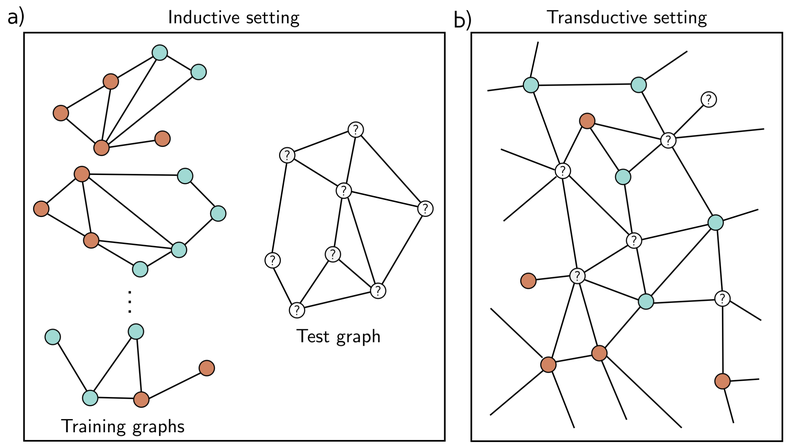
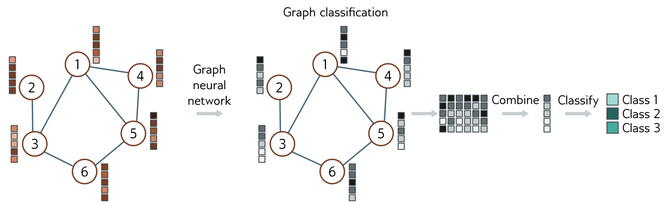


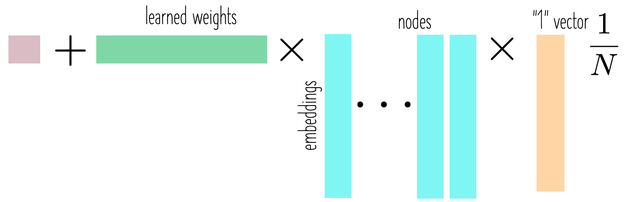
Figure 13.8 Inductive vs. transductive problems. a) Node classification task in inductive setting. We are given a set of I training graphs, where the node labels (orange and cyan colors) are known. After training, we are given a test graph and must assign labels to each node. b) Node classification in transductive setting. There is one large graph in which some of the nodes have labels (orange and cyan colors) and others are unknown. We train the model to predict the known labels correctly and then examine the predictions at the unknown nodes.

Inductive learning	Transductive learning
☺ We learn the rule that maps the graph (or its elements) to a target output, then apply this rule to new data.	☹ It does not produce a rule, but merely a labeling for all of the unknown outputs.
☹ Cannot access the original features of the testing samples during the training stage.	☺ Considers both the labeled and unlabeled data at the same time. ☺ It has the advantage that it can use patterns in the unlabeled data to help make its decisions.
☺ No need to store the testing nodes and their original features in memory.	☹ High memory cost as it jointly stores information for training and testing node. ☹ Higher computational time.
☺ Test the trained model on unseen nodes (new ones).	☹ When extra unlabeled data (new testing samples) are added, the model should be retrained from scratch.

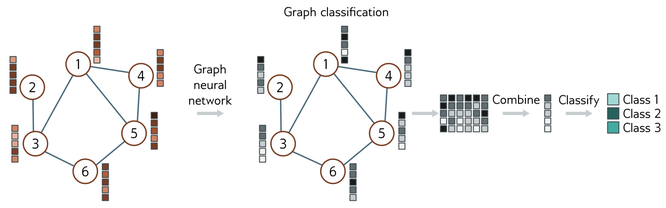
Inductive ~~node~~ graph classification



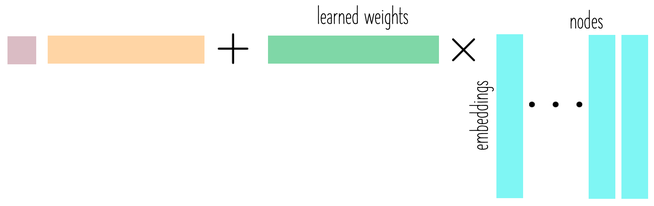
$$y = \text{sig}\left[\beta_K + \boldsymbol{\omega}_K \mathbf{H}_K \mathbf{1}/N\right]$$



Transductive node classification



$$y = \mathbf{sig}[\beta_K \mathbf{1}^T + \omega_K \mathbf{H}_K]$$



$\mathbf{sig}(\cdot)$ applies the sigmoid function independently to every element of the row vector input.

Question for reflection: How to batch nodes in a transductive learning paradigm?

Personal reflections and notes